

Silent Consent, Persistent Risk: Android Permission Groups and Custom Permissions

Olawale Amos Akanji^[0009-0005-7419-5108], Manuel Egele, and Gianluca Stringhini

Boston University, Boston MA 02215, USA
{olawalea, megele, gian}@bu.edu

Abstract. Android’s permission system is designed to balance usability with informed consent, yet two legacy mechanisms still undermine that balance in Android 16: (i) permission groups that silently auto-grant new permissions within a group after a user’s initial approval, and (ii) **normal**-level custom permissions that are auto-granted at install and enable cross-app access with no user visibility. We conduct a longitudinal analysis of 19.3 million APKs spanning 5.97 million unique apps (distinct package identifiers) from the AndroZoo repository, combined with on-device validation on Android 16. Among 2,244,575 multi-version apps, 381,026 (17%) silently gain permissions within already-granted groups. Using VirusTotal detections with primary threshold $t = 20$, apps flagged as malware expand within groups at a higher rate than benign apps (odds ratio = 1.35, $p < 0.001$); the association holds across every tested threshold and concentrates in permission-heavy apps (OR = 2.06 in the top quartile). We also identify 307 cross-developer **normal**-custom-permission pairs that expose contacts, SMS, location, authentication credentials, user identity, and medical records to unrelated apps without any user prompt. A lightweight prototype built on public Android APIs recorded 23 silent expansion events across 13 apps during a 96-day single-device pilot, showing that update-time transparency is reachable without OS modification. Our results show that consent erosion persists despite a decade of platform hardening and affects apps ranging from obscure utilities to widely deployed and pre-installed software.

Keywords: Android Security · Mobile Privacy · Cross-App Data Exposure · Permission Systems · Permission Groups · Custom Permissions · Informed Consent

1 Introduction

Android powers over three billion active devices [33], and its permission system governs how apps access sensitive device resources and user data, including contacts, location, camera, microphone, and storage. Apps declare the permissions they require in their manifest, and the system enforces access control at runtime. Permissions fall into three protection levels: **normal** permissions (e.g., internet

access) are auto-granted at install because they pose minimal risk; **dangerous** permissions (e.g., reading contacts) require explicit user consent because they access private data; and **signature** permissions are granted only to apps signed with the same developer key as the app that defined them. This model gives users visibility into what resources apps can access and, for dangerous permissions, the ability to grant or deny access.

Modern permission systems must balance two competing goals: protecting users from privacy violations while enabling seamless application interaction. Android embodies this tension in its permission architecture. Early versions (Android 1.0–5.1) relied on install-time permission lists that users largely ignored, leading to “symbolic consent” [11, 28, 29, 35]. Android 6.0 addressed this by introducing runtime permissions: apps must now request each dangerous permission individually at the moment the protected resource is needed, and users see a contextual dialog they can grant or deny.

Despite this shift, two mechanisms inherited from Android 1.0 remain active in Android 16 and continue to weaken the consent guarantees that runtime prompts were designed to provide: **permission groups** and **normal-level custom permissions**.

Permission Groups. Android clusters related permissions into named groups to reduce the number of runtime prompts shown to users [15]. Once a user grants any permission from a group, the system silently auto-grants (i.e., automatically approves without showing a prompt) subsequent additions to that same group during app updates, enabling *silent capability expansion* without renewed consent.

Normal-Level Custom Permissions. Android allows developers to define proprietary permissions that guard exported components [30, 38]. When declared with protection level **normal** (the default), any requesting app, regardless of developer identity, receives automatic approval at install time with no user prompt and no indication in Settings, creating an *implicit permission delegation* path that bypasses Android’s dangerous-permission enforcement. Section 2 examines the concrete security tensions and attack scenarios these mechanisms enable.

Prior work has documented confused-deputy attacks and custom-permission misuse enabling cross-app leakage [26, 30, 38, 50]: Calciati et al. [26] measured permission-group auto-grant prevalence on 2,865,553 app versions, and Gamba et al. [30] analysed **normal**-level custom-permission exposure over 2.2M apps. Both mechanisms persist in Android 16 substantially unchanged. We revisit them at ecosystem scale, validate their exploitability on Android 16, and evaluate a lightweight public-API transparency intervention.

We investigate four research questions:

- **RQ1.** To what extent do permission groups enable silent expansion of app capabilities across updates without renewed user oversight?
- **RQ2.** Is silent permission-group expansion an ecosystem-wide consent failure, or is it disproportionately an attribute of malware rather than benign apps?
- **RQ3.** How prevalent are **normal**-level custom permissions that enable cross-developer data access, what sensitive data do they expose, and can we confirm active exploitability on a current Android release?

- **RQ4.** Can lightweight, interface-level interventions restore visibility into update-time permission changes without compromising usability?

To address these questions, we re-estimate both mechanisms on Android 16 at ecosystem scale (19,324,760 APKs from 5,965,583 unique apps in AndroZoo, deduplicated by package name), compare expansion rates between VirusTotal-flagged and benign apps under a principled sensitivity sweep, validate `normal`-custom-permission re-delegation on-device through per-pair Probe Apps, attribute the recovered exploitation surface to app-core code versus bundled third-party libraries, and release a runnable update-time transparency prototype built on public Android APIs. Our contributions are:

1. We show that silent permission-group expansion remains a systemic consent failure on Android 16: 17% of multi-version apps in the AndroZoo corpus silently gain permissions within already-granted groups, and on-device testing confirms that the auto-grant mechanism still fires across every dangerous group on Android 16 with no runtime dialog or Settings indication.
2. We show that this silent expansion is not a uniformly latent design weakness but is preferentially engaged by malware: the odds-ratio association is statistically significant at every tested VirusTotal threshold in the Zhu et al. [55] range and, after stratifying by permission breadth, concentrates in permission-heavy apps (OR = 2.06 in the top quartile).
3. We identify 307 cross-developer `normal`-custom-permission pairs that re-delegate contacts, SMS, location, authentication credentials, user identity, and medical records, and confirm active exploitability on Android 16 by instrumenting Probe Apps that retrieve the sensitive payloads; call-site attribution assigns the majority of the recovered surface to bundled third-party libraries rather than to app-core code.
4. We implement a public-API update-time transparency prototype and validate it in a 96-day Pixel 7 deployment: 23 silent expansion events across 13 apps, including apps with over one billion installs, at roughly one notification every four days, showing that lightweight interface-level interventions can restore update-time consent visibility without OS modification.

2 Motivation

Despite Android’s shift to runtime permissions and richer controls, two legacy mechanisms, permission groups and `normal`-level custom permissions, continue to undermine informed consent. Permission groups allow silent capability expansion within already-approved groups during app updates, while `normal` custom permissions enable cross-app access to guarded components without any user prompt or runtime dialog.

2.1 Google’s Justification: Prompt Fatigue and Openness

Android’s permission grouping model minimizes repeated dialogs by auto-granting future permissions within a group once any member is approved, framing this as a response to prompt fatigue [15].

For custom permissions, the rationale is interoperability. Developers define proprietary permissions to guard exported components [7], each carrying a protection level: **normal** (auto-granted at install, the default), **signature** (same-key apps only), or **dangerous** (runtime user consent). The **signature** level restricts grants to apps signed with the same key, though key sharing across organizations weakens this guarantee [30]. By contrast, **normal**, the default when developers omit the level, provides no protection: any requesting app receives automatic approval regardless of developer identity, with no prompt and no Settings entry. Gamba et al. [30] documented real apps exposing private data via **normal** custom permissions in a responsible disclosure; Google acknowledged the issue but framed it as a consequence of Android’s openness [30], leaving a recognized yet unresolved attack surface.

2.2 The Security Tension in Modern Android

Permission groups and custom permissions each introduce distinct security tensions that persist on modern Android. We examine three facets: risk asymmetry from forced bundling within groups, granularity that the platform offers at grant time but hides at revocation, and the trust boundary that normal-level custom permissions silently bypass.

Risk Asymmetry and Forced Bundling. Despite recent granularity improvements (Android 13 split media access into separate `READ_MEDIA_IMAGES`, `READ_MEDIA_VIDEO`, and `READ_MEDIA_AUDIO` [10]; Android 14 added a partial photo-picker [14]), many permission groups still bundle capabilities with fundamentally different risk profiles. For example, a two-factor authentication app that requests `READ_SMS` to extract one-time passcodes may later add `SEND_SMS` in an update. Because both belong to the SMS group, Android auto-grants the addition without any prompt. If the app is compromised or acquired by a malicious actor, it can silently dispatch premium-rate messages, a qualitative expansion from passive read access to active send capability that operates entirely within documented group semantics.

Granularity Without Revocation. Android has introduced fine-grained prompts but not equally fine-grained revocation, creating a one-way ratchet: permissions can be silently widened through updates, but users cannot individually narrow them. Media permissions illustrate the gap: an app requesting only `READ_MEDIA_IMAGES` triggers a single photo-access prompt [47], but if a later update adds `READ_MEDIA_VIDEO`, Android auto-grants it silently. As Figures 1a and 1b show, Settings provides only a single “Photos and videos” toggle. Revoking video necessarily revokes photos, forcing users to choose between over-permissioning and losing functionality entirely [45, 52].

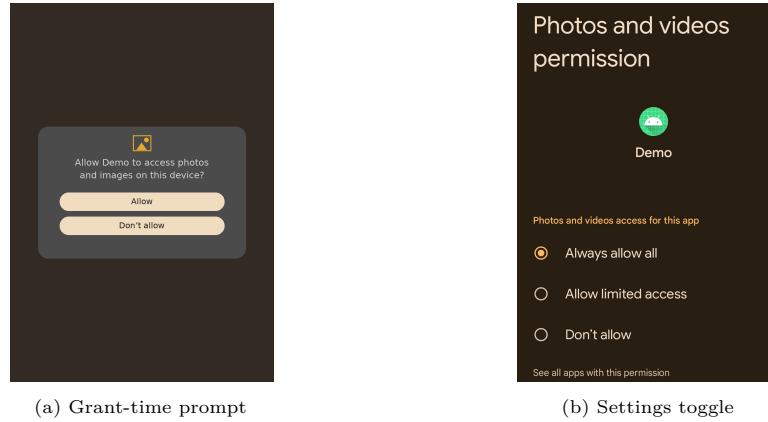


Fig. 1: Android 16 media permissions: (a) the runtime dialog offers a granular prompt for `READ_MEDIA_IMAGES`, but (b) Settings collapses all media permissions into a single “Photos and videos” toggle, erasing prompt-level granularity. `READ_MEDIA_IMAGES` is a standalone permission that excludes video and audio; a user who granted only photo access has no way to revoke video access independently because Settings does not expose the per-permission granularity that the runtime prompt created.

The Limits of “Openness” as Defense. Permission groups at least provide a one-time consent decision; the problem is that silent updates can expand capabilities without renewed approval. `normal`-level custom permissions provide no user-facing consent at all. A developer who defines a custom permission must declare four manifest attributes (`android:name`, `android:protectionLevel`, `android:label`, and `android:description`), explicitly signaling intent to restrict access. Yet the default protection level (`normal`) nullifies that intent: every requesting app receives automatic approval at install time, with no prompt and no Settings entry. We refer to the app that defines a `normal`-guarded exported component as the *exploitable app*, and the unrelated third-party app that declares the same permission name to reach that component as the *exploiting app*.

Our threat model is an exploiting app from a different developer that declares the exploitable app’s `normal` custom permission and invokes exported components guarded by it. For example, a health-tracking app storing medical records via a `normal`-guarded provider can have its data silently read by any other app that declares the same permission name, without the exploitable developer’s knowledge, turning interoperability into an unmediated delegation channel.

3 Methodology

We combine longitudinal static analysis of 19.3M APKs (5.97M unique apps, deduplicated by package name) spanning 2013–2025 with temporal analysis of permission-group expansion trends aligned to major Android policy changes, malware vs benign comparison using VirusTotal classification, cross-developer custom

permission linking through bytecode inspection of sensitive data flows, dynamic validation on Android 16 devices, and a 96-day deployment of a transparency prototype built using only public APIs.

3.1 Dataset

We analyse 19,324,760 APKs from 5,965,583 unique packages in AndroZoo [2] spanning Google Play and alternative markets, extracting requested permissions, custom permission definitions, protection levels, and exported components from each `AndroidManifest.xml`.

3.2 Permission Groups: Longitudinal Analysis and Dynamic Testing

We perform a fully automated longitudinal analysis over the corpus using Androguard [5] to parse `AndroidManifest.xml` from every APK and extract its `<uses-permission>` declarations. For each package with at least two versions, we compare adjacent versions, map every declared permission to its Android Open Source Project (AOSP) protection group, and flag cases where the later version introduces a permission whose group was already present in the earlier version; under Android’s runtime model, once any member of a group has been granted, additional members from that group are implicitly approved.

We also aggregate expansion activity by release year to determine whether major platform changes reduced prevalence or merely shifted which groups are affected (Figure 2).

To assess whether silent permission-group expansion is disproportionately associated with malware, we label each app using VirusTotal (VT) detection counts, that is, the number of antivirus engines that classify a given APK as malicious when it is submitted to the VirusTotal aggregator. Zhu et al. [55] show that threshold-based aggregation is meaningful over the range $2 \leq t \leq 39$, with thresholds below 15 offering the strongest precision–recall trade-off. We use $t = 20$ as the primary threshold and report sensitivity at $t \in \{2, 5, 10, 39\}$ to span the admissible range.

For each app we record whether it ever exhibited a permission-group expansion, producing a 2×2 table of expanding vs. non-expanding apps crossed with malware vs. benign labels. From this table we compute the odds ratio (OR) [39], where values > 1 indicate positive association, a Pearson chi-squared test for significance [46], and a Mantel–Haenszel stratified estimate [41] that controls for the total number of permissions each app declares.

Finally, we verify that these silent expansions actually occur on a current device. On physical Android 16 devices we install minimal test apps: a baseline version requests a single group permission (e.g., `READ_CONTACTS`); a follow-up version adds another from the same group (e.g., `WRITE_CONTACTS`). After granting the baseline, we update in place and observe whether Android surfaces a new runtime dialog or silently enables the added capability. We also inspect Settings to determine whether revocation is exposed per permission or only via a group-level

toggle. This pairing of longitudinal analysis with device-level observation lets us measure prevalence while establishing behavioral ground truth on Android 16.

3.3 Custom Permissions: Detection, Linking, and Validation

We implement the custom-permission pipeline as a fully automated corpus-scale analysis,¹ using Androguard [5] for manifest and DEX parsing. Each APK passes through three static stages: manifest classification and component-to-permission binding, bytecode-level data-sensitivity analysis of the declaring side, and call-site resolution on the requesting side. These stages produce candidate pairs, which we then filter to those signed by different developer certificates and validate end-to-end on an Android 16 device.

Manifest classification. We classify every `<permission>` element against the canonical Android Open Source Project (AOSP) permission list parsed from successive Android releases; declarations outside that list are labeled custom, and we record the `android:protectionLevel` (defaulting to `normal` when omitted). We then resolve each custom permission to the exported component it guards (`<provider>`, `<activity>`, `<service>`, or `<receiver>`) and restrict to attachments that are simultaneously `protectionLevel="normal"` and `exported="true"`, since same-process guards and signature-protected components cannot be re-delegated across developer boundaries.

Bytecode-level data-sensitivity analysis. Manifest matching alone yields many benign candidates whose guarded components expose only app configuration or user-interface state. We therefore analyse the DEX bytecode of each declaring component for evidence that it returns AOSP-protected data, following prior DEX-level Android inspection work [1, 27]. For every declaring component we use Androguard’s [5] method- and basic-block-level analysis layer to walk return paths: for content providers we resolve `getType()` and trace `ContentProvider.query()` return values back to sensitive sources such as `ContactsContract` and `Telephony.Sms`, and extract the column-name string constants they expose (e.g., `PHONE_NUMBER`, `USER_EMAIL`); for activities and receivers we trace payloads set via `setResult()`; for services we resolve method signatures returned by `onBind()` [30].

Call-site resolution on the requesting side. A bare `<uses-permission>` declaration is insufficient evidence of access: prior measurement work on Android has consistently had to recover the actual invocation sites before drawing behavioural conclusions, whether for permission re-delegation [30, 50], covert data access [48], or taint tracking of sensitive sources [27]. For every app that declares a custom permission matching a sensitive provider, we use Androguard’s [5] DEX analysis layer to enumerate each method’s invocation list and flag calls to `ContentResolver.query/insert/update/delete/call`; we then resolve the authority-URI argument via constant propagation through `Uri.parse`, `String`

¹ Artifact availability: the analysis scripts, Probe App harness, AutoGrant Monitor APK artifact, and aggregate per-group outputs are available at <https://github.com/marshallwahlexyz1/Silent-Consent-Persistent-Risk-Android-Permission-Groups-and-Custom-Permissions.git>.

concatenation, and `StringBuilder` chains, and accept the call site only when the resolved authority matches the target provider. For each matched call we record the enclosing method’s fully qualified declaring class so that downstream app-core vs. third-party library attribution can be performed directly on the bytecode-observable call graph. Because custom-permission names, provider authorities, and `android:permission` manifest attributes are runtime-load-bearing strings and must survive identifier renaming to function at all, manifest-level pair identification is obfuscation-stable; call-site enumeration is the only stage where ProGuard repackaging reduces recall, and since every reported pair is anchored on an observed call site, obfuscation cannot inflate precision. It can only cause us to miss pairs whose exploiting call site has been renamed beyond recognition, making the reported set a lower bound.

Runtime validation. Because static analysis establishes only that a cross-developer data path exists, we validate each pair whose provider exposes named columns in bytecode via an on-device experiment on Android 16. For each such pair we instrument both halves. On the exploitable side we install the declaring app on the device and grant its AOSP dangerous permission (e.g., `READ_CONTACTS`); the app queries the corresponding system provider and stores the returned rows inside its own exported `ContentProvider`, which remains guarded only by the `normal` custom permission. On the requesting side we build a minimal *Probe App* that declares that custom permission via `<uses-permission>` (auto-granted at install with no user prompt) and issues a `ContentResolver.query` against the exploitable app’s provider authority using the statically resolved projection. A pair is confirmed only when (i) the Probe App reads back a non-empty `Cursor` with valid field values from the exploitable app’s provider, and (ii) the Android `AppOps` journal (the platform’s per-operation access log) records a protected operation such as `OP_READ_CONTACTS` attributed to the exploitable app, confirming that the sensitive data behind the `normal` guard originated from an AOSP dangerous-permission read rather than from test fixtures.

3.4 Prototype: Surfacing Silent Expansions to Users

We implement a lightweight prototype that detects silent permission expansions without OS modification. The prototype registers for package lifecycle broadcasts (`ACTION_PACKAGE_ADDED`, `ACTION_PACKAGE_REPLACED`), extracts each app’s declared permissions at update time, and compares them against a cached snapshot from the previous version. It then maps newly introduced permissions to their groups and triggers a notification only when a new permission appears within an already granted group. To reduce noise, we deduplicate per app/version, ignore first-time group introductions, and include a direct link to the app’s permission settings. This provides update-time visibility with minimal user interruption while relying only on public APIs.

Table 1: Top-6 permission-group expansion flows (main groups).

Group	Flow	Count	Group	Flow	Count
CONTACTS	Get Accts → Read	13,164	PHONE	Read State → Call	26,430
CONTACTS	Read → Get Accts	5,830	PHONE	Call → Read State	9,674
CONTACTS	Read → Write	4,747	PHONE	Read State → Read Num.	6,115
CONTACTS	Get Accts → Write	4,299	PHONE	Read State → Process Out.	4,605
CONTACTS	Write → Get Accts	1,999	PHONE	Call → Read Num.	2,928
CONTACTS	Write → Read	552	PHONE	Call → Process Out.	2,306
SMS	Send → Receive	2,186	CALL LOG	Read → Write	206
SMS	Send → Read	2,152	CALL LOG	Write → Read	43
SMS	Receive → Read	1,302			
SMS	Receive → Send	1,236			
SMS	Read → Send	1,057			
SMS	Read → Receive	1,007			

4 Results

We first present the prevalence, temporal evolution, and malware association of permission-group expansion. We then turn to cross-developer exploitation through `normal` custom permissions and report the deployment results of the update-time transparency prototype.

4.1 Permission Groups: Longitudinal Prevalence and Dynamic Confirmation

Longitudinal Analysis. Of 2,244,575 multi-version apps, 381,026 (17.0%) silently gain at least one permission inside an already-granted group, contributing 928,452 auto-granted additions in total, or 2.44 additions per expanding app on average. Table 1 lists the dominant observed flows for the CONTACTS, SMS, PHONE, and CALL_LOG groups, while Appendix Table 4 reports the corresponding top flows for the remaining five groups. This rate is of the same order as the earlier Calciati et al. [26] measurement,² indicating that the mechanism has persisted on modern Android.

Per-group expansion volume (Appendix B, Table 5) shows that STORAGE, LOCATION, and PHONE account for 94.5% of all expansion cases. The communication groups, SMS and CALL_LOG, contribute a much smaller share of total expansions but, as we show below, have the highest concentration of malware-flagged expanders.

We next test whether the aggregate rate could be inflated by AndroZoo’s multi-market collection model. Using the AndroZoo `markets` metadata, we stratify expanding apps by market provenance and separate same-market histories from cross-market histories. The effect remains visible in the Play-only and non-Play-only buckets and is not concentrated in cross-market package histories, which indicates that the result is not driven primarily by market-specific version mixing.

² Cross-study rate comparisons should be treated as approximate because datasets, years, and denominators differ.

Temporal Trends and Platform Evolution. Examining ecosystem evolution shows that expansion behavior varies across periods. Based on each app’s release date, Figure 2 summarizes yearly trends. Panel (a) shows that the number of unique apps exhibiting expansion peaked around 2016 (~22,000 apps) and has declined sharply in recent years (~500–2,700 apps from 2022–2024). Panel (b) shows that the expansion rate (average expansions per expanding app) remained relatively stable (1.3–1.9) through 2021 but increased sharply to 3.5+ in 2023–2024. This divergence indicates that while fewer apps now expand permissions, those that do are adding more permissions per update. We interpret these trends in light of two kinds of platform change: changes that alter group composition directly, and adjacent privacy changes that leave within-group auto-grant semantics intact.

Group-affecting changes.

- **CALL_LOG separation from PHONE (2018).** Android 9 split `READ_CALL_LOG`, `WRITE_CALL_LOG`, and `PROCESS_OUTGOING_CALLS` into a new `CALL_LOG` group [8]; Google Play further restricted SMS/Call Log access to default handler apps [32]. SMS/CALL_LOG expansions drop sharply by 2020 (155 events, –80.0% vs 2017).
- **NEARBY_DEVICES group (2021–2022).** Android 12 introduced a `NEARBY_DEVICES` group for Bluetooth [13]; Android 13 added `NEARBY_WIFI_DEVICES` [12]. Expansions remain low (≤ 44 /year).
- **Media granularity (2022).** For API 33+, Android 13 replaces `READ_EXTERNAL_STORAGE` with the granular media permissions `READ_MEDIA_IMAGES`, `READ_MEDIA_VIDEO`, and `READ_MEDIA_AUDIO` [6, 12]. The legacy permission nevertheless remains in `STORAGE`, creating a reverse expansion path: an app granted only `READ_MEDIA_IMAGES` can silently gain `READ_EXTERNAL_STORAGE` in a later update. `STORAGE` expansions dip in 2022 (1,073 vs 2,985, –64.1%) but spike in 2023 (8,159).
- **Background body sensors (2022).** Android 13 added `BODY_SENSORS_BACKGROUND` [12]. `SENSORS` expansions remain rare (0.3% of events).

Adjacent changes. Runtime permissions (Android 6.0) [21], one-time grants and auto-reset (Android 11) [16, 19], and system pickers (SAF [9], Contact Picker [18], Photo Picker [17, 20]) strengthen consent in complementary ways but do not alter within-group auto-grant semantics. Users still receive no notification when an app adds a permission to an already-approved group. The temporal analysis therefore points to reactive, per-group policy responses that redraw the boundaries of silent expansion without addressing the underlying mechanism.

Dynamic Confirmation on Android 16. We verify these patterns on Android 16 (API 36) through controlled experiments. For each of the nine dangerous permission groups, we install paired test applications: the baseline version requests a single group member (e.g., `READ_MEDIA_IMAGES`); the update adds an additional permission from the same group (e.g., `READ_MEDIA_VIDEO`). After granting the baseline permission and updating in place, Android 16 automatically grants every newly introduced group permission in all nine groups: no runtime dialog, no

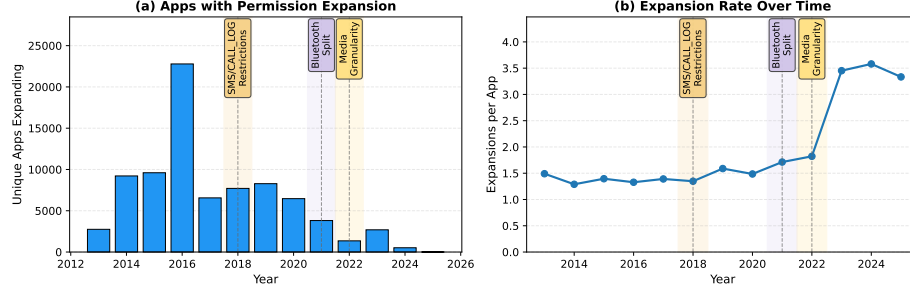


Fig. 2: Permission group expansion trends over time (2013–2025) with major Android platform changes annotated. (a) Number of unique apps exhibiting at least one permission-group expansion per year. (b) Expansion rate (average expansions per expanding app) per year.

update notification, and no distinction in Settings. These experiments confirm that the longitudinal expansion patterns represent real, silent capability growth on the current platform. These results answer **RQ1**: permission groups enable widespread silent expansion—17% of multi-version apps gain permissions within already-granted groups—and the auto-grant mechanism remains active across all nine dangerous groups on Android 16.

4.2 Malware vs Benign Apps

We compare how often malware and benign apps use permission-group expansion to determine whether the mechanism is primarily a malware tactic or a broader ecosystem problem. Using VirusTotal labels (Section 3), we classify each app by its highest detection score across all versions, with $t = 20$ as the primary threshold.

Per-app, malware-flagged apps expand at a higher rate than benign apps. We call an app *malware-flagged* at threshold t when at least t of the VirusTotal antivirus engines classify any version of that app as malicious. At $t = 20$, 1.63% of the 381,026 expanding apps are malware-flagged, compared with 1.29% of the 2,244,575 multi-version baseline. The resulting odds ratio (OR) is 1.35 ($\chi^2 = 430.9$, $p < 0.001$), so malware-flagged apps are more likely than benign apps to have silently expanded within a permission group. Figure 3 shows that this is not a one-threshold artifact: the association remains statistically significant at every tested value, with OR = 1.38 at $t=2$, 1.31 at $t=5$, 1.24 at $t=10$, 1.35 at $t=20$, and 2.67 at $t=39$ (all $p < 0.001$). In absolute terms, however, most expanding apps are still benign (98.37% at $t = 20$) because benign apps dominate the ecosystem numerically.

One possible explanation for this gap is that malware simply requests more permissions overall and therefore has more opportunities to expand. We test that possibility by stratifying apps by total declared permission count into four quartiles and recomputing the association (Table 6). The stratified OR is 1.05

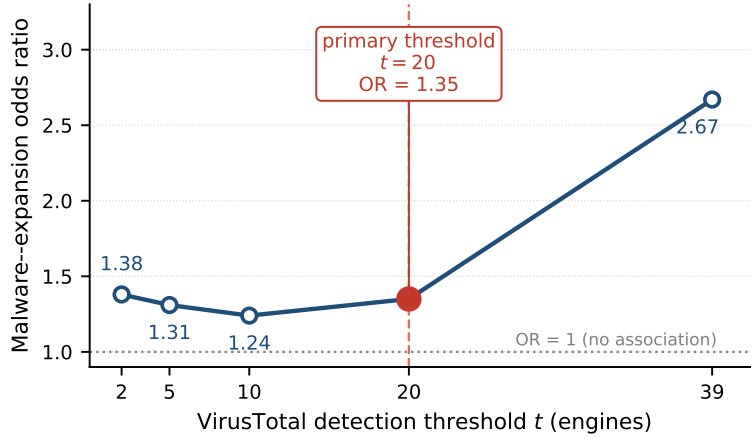


Fig. 3: Malware-expansion odds ratio across the five reported VirusTotal thresholds (primary threshold $t = 20$ highlighted).

(95% CI [1.02, 1.08], $p = 0.001$), which is statistically significant but substantially attenuated. Permission breadth therefore explains most of the crude association. The remaining effect is concentrated in the top quartile (≥ 24 permissions), where malware expands at about twice the rate of benign apps (OR = 2.06), while the three lower quartiles show comparable rates.

Not all permission groups carry equal malware risk. Decomposing the 381,026 expanding apps by group and overlaying the VT labels, the communication groups concentrate the highest malware share (SMS: 72.6%, CALL_LOG: 54.3%), while the high-volume groups STORAGE and LOCATION contribute the largest absolute malware counts despite lower within-group percentages. To characterise what “malware-flagged” actually covers in our corpus, we retrieve per-engine family labels for all 6,225 apps that cross the $t = 20$ threshold: the flagged bucket is a mixture of adware (36.1%), trojans/malware (36.0%), PUA/riskware (22.2%), and other categories (5.6%), so the odds-ratio association we report holds against a broad malware population rather than being driven by a single family. These results answer **RQ2**: silent expansion is an ecosystem-wide consent failure, but malware-flagged apps engage it at a higher per-app rate, with the association concentrated in permission-heavy apps.

4.3 Custom Permissions: Evidence of Real-World Exploitation

We investigate three aspects of **normal**-level custom permissions on Android 16: (1) prevalence and component types guarded, (2) whether those components handle sensitive user data, and (3) whether apps from unrelated developers exploit these permissions to access that data without user awareness.

Prevalence of Normal Custom Permissions. Across the corpus we extract 2,630,134 distinct custom permission names (Table 2). Most use **signature** protection (90.4%), while 9.4% are defined as **normal** and are therefore auto-granted to any requesting app. Table 2 also breaks the **normal** subset down by the type of exported component each permission guards. Of these, only the 18,475 permissions attached to content providers expose persistent structured user data in our pipeline. In our sample, exported activities, services, and broadcast receivers return user-interface state, remote procedure call (RPC) primitives, or event broadcasts and, under our bytecode filter for AOSP-protected sources, yield no cross-developer pairs in which the exploitable app demonstrably returns sensitive fields. We therefore focus the remainder of this section on **normal**-level **<provider>** guards.

Table 2: Custom permissions: protection-level split (left) and, for the **normal** subset, guarded component type (right).

Protection level	Count	%	Component (normal)	Count	%
signature	2,378,863	90.4	<provider>	18,475	7.5
normal	246,958	9.4	<activity>	87,527	35.4
dangerous	3,853	0.1	<service>	4,005	1.6
Other	461	<0.1	<receiver>	4,743	1.9
			Unattached	132,208	53.5
Total	2,630,134	100.0	Total	246,958	100.0

Cross-Developer Pairs. Among the 18,475 **normal**-guarded providers, we focus on those that constitute an *active* cross-developer exploitation surface, that is, providers for which an unrelated app both declares the matching permission and actually calls into it. We therefore filter this set to pairs in which (i) an *exploitable app* declares the **normal** custom permission and exposes the guarded **ContentProvider**, (ii) an *exploiting app* declares the same permission name and issues a **ContentResolver** **query/insert/update/delete/call** against that authority, and (iii) the two APKs are signed with different developer certificates. This filter yields 307 active cross-developer pairs over 323 unique packages: 83 appear only as exploitable apps, 79 only as exploiting apps, and 161 in both roles across different pairs. These apps expose data through one **normal** permission while accessing another app’s data through a different one, so the exploitation surface is bidirectional.

We characterize the 323 unique packages along three axes: certificate provenance, Google Play installation tiers, and, for the requesting side, whether the exploiting **ContentResolver** call originates from app-core code or from a bundled library.

Grouping the 323 APKs by the public key of their v2+ APK signing block, 228 are signed by third-party developer keys, 49 by original equipment manufacturer (OEM) platform keys (Samsung, OPPO, Motorola, Amazon, Microsoft), and

46 by Google platform keys; 74 of the 307 statically confirmed active cross-developer pairs have at least one OEM- or platform-signed APK on either side. Cross-referencing the same 323 packages against live Google Play metadata, 102 are currently listed on Play and 221 are not (removed, region-restricted, or only shipped through OEM or alternative channels). Within the 102 Play-listed packages, 24 are in the $\geq 100\text{M}$ -install tier: 12 appear only as exploitable apps, 4 only as exploiting apps, and 8 in both roles across different pairs. These results show that the cross-developer exploitation surface is not confined to obscure third-party apps; it includes OEM- and platform-signed packages and Play-distributed apps with large installation counts on both the defining and requesting sides.

For each exploiting `ContentResolver` call site, we attribute the declaring class either to the exploiting app’s own code or to a bundled third-party library, following prior Android work that recovers embedded libraries and uses them to study third-party code behavior in the wild [22, 25, 34, 36, 37, 40]. We classify a call site as *app-core* when the declaring class shares a package prefix with the exploiting app’s manifest package, as *third-party library* when it matches a known SDK package prefix, and as *unclassified* otherwise. ProGuard repackaging primarily moves third-party code into the unclassified bucket rather than into app-core, so the resulting counts are a conservative lower bound on SDK involvement. To cross-validate the package-prefix heuristic, we ran LibScout [22] profile matching against 8,542 pre-computed library signatures on the 329 pair APKs: 284 of 315 successfully analysed APKs (90.2%) contain at least one signature-level library match (mean 24.0 libraries per APK), confirming that bundled third-party SDK code is pervasive in the pair corpus. Of the 273 pairs with a recoverable exploiting call site, 126 (46%) resolve exclusively to third-party library code, 127 (47%) contain both app-core and third-party call sites, 10 (4%) resolve exclusively to app-core, and the remainder are unclassified.

For 40 of the 307 statically confirmed active cross-developer pairs, bytecode analysis of the exploitable provider recovers readable column-name string constants, so only these pairs admit both field-level naming and a reproducible on-device probe; 39 of the 40 are still installable on Android 16, and we exclude the remaining pair.

For each of the 39 dynamically validated pairs on Android 16, we compile a *Probe App* that reproduces the exploiting call site while holding no dangerous permissions and execute it against the exploitable provider. All 39 probes retrieve the sensitive payload via `ContentResolver.query`, and the Android `AppOps` journal records a protected operation, confirming that the `normal` custom permission functions as a runtime re-delegation channel on Android 16 for the subset amenable to field-level probing. On this dynamically validated subset, the same attribution yields 18 SDK-only, 19 mixed, 1 app-core-only, and 2 unclassified.

For the remaining 267 statically confirmed pairs, we confirm the presence of a persistent structured store (201 SQLite-backed, 62 file-backed) but cannot recover field names or run an equivalent on-device probe. We therefore report 307 as the statically confirmed active cross-developer surface and restrict field-level and dynamic claims to the 39 pairs validated on Android 16. Table 3 reports the

exposed data categories for that 39-pair subset, with each pair assigned to its primary category so the pairs are counted exactly once.

Categories of Real-World Exploitation. Whether these flows are intentional interoperability or accidental exposure is not a defense: in both cases, user data crosses developer boundaries without a runtime prompt, Settings visibility, or explicit user authorization. Table 3 reports, for the 39 probed pairs, the primary data category returned by the exploitable provider and whether that category is also gated by an AOSP dangerous permission. Type A categories duplicate AOSP-gated data through a `normal`-only channel (so a `normal`-permission holder reads data that the AOSP gate would otherwise prompt for); Type B categories are app-specific records that no AOSP permission covers, so the `normal` permission is the *only* access control.

Table 3: Primary data category assigned to each of the 39 probed pairs. Type A: also gated by an AOSP dangerous permission. Type B: no AOSP gate; the `normal` permission is the sole access control.

Type	Data Category	Pairs	AOSP Gate (Type A only)
A	Contact data	12	<code>READ_CONTACTS</code>
A	Authentication credentials	8	<code>GET_ACCOUNTS</code>
A	User identity	7	<code>READ_PHONE_STATE</code>
A	Location	2	<code>ACCESS_FINE_LOCATION</code>
A	Messages	1	<code>READ_SMS</code>
B	File and storage paths	5	—
B	Medical / health records	1	—
B	Financial data	2	—
B	Settings / config	1	—
Total category instances		39	

These results answer **RQ3**: `normal`-level custom permissions are widespread (246,958 definitions, 9.4% of all custom permissions), and among the 18,475 that guard content providers, 307 statically confirmed active cross-developer pairs expose contacts, SMS, location, credentials, identity, and medical records to unrelated apps. Of these, 39 dynamically validated pairs on Android 16 are confirmed exploitable on-device, and the majority of recoverable exploitation call sites originate from bundled third-party SDKs rather than app-core code.

4.4 Prototype Deployment: Feasibility of Update-Time Transparency

The prototype (Section 3) runs on a Google Pixel 7 with Android 16 and 214 installed applications spanning social media, finance, and utilities. Over a 96-day deployment it detected 23 silent expansion events across 13 apps, none of which surfaced any indication to the user that a new permission had been silently granted. The affected apps include widely-used third-party apps in social media (1B+ installs), short-video (1B+ installs), photo sharing (1B+ installs), and

finance (10M+ installs) categories, as well as OEM/platform software responsible for connectivity, device diagnostics, and accessibility services. We report these in aggregate rather than by package name because disclosure remains active. Each expansion event triggers a notification such as “*This app has gained a new permission: **Read Images**. Review in Settings.*” Without that notification, the new permission is silently active, with no consent dialog, no visual cue, and no Settings entry. The notification restores informed decision-making by letting the user revoke the group, keep the permission, or uninstall the app, though as discussed in Section 2 Android exposes only a group-level toggle, so revoking one unwanted permission means losing the entire group.

Notification burden remains low: 23 events over 96 days amounts to roughly one notification every four days. Our longitudinal data show a median rate of 1.0 within-group additions per app per year; even if every installed app expanded once per year, a user with 80 apps [24] would receive about 1.5 notifications per week, while a power user with 365 apps would receive roughly one per day. This is well below the dozens of daily social-media alerts most users already manage, and each notification concerns a genuine capability change. These results answer **RQ4** as a feasibility existence proof: on a single device and install profile, lightweight, interface-level interventions can restore update-time transparency without imposing a per-day user burden; ecosystem-scale deployment and user-perception studies remain future work.

5 Discussion

Our measurements show that Android’s response to both mechanisms has been selective rather than systematic: the platform has repeatedly added user-facing controls when individual risks became visible (call log, Bluetooth, broad media access), yet update-time group expansion and **normal**-level custom-permission re-delegation still bypass consent at ecosystem scale on Android 16. For permission groups, the implication is straightforward: within-group additions should trigger risk-sensitive re-consent or, at minimum, explicit update-time disclosure and revocation controls that preserve the granularity offered at grant time. Our 96-day prototype shows that such disclosure is implementable on top of public APIs with a manageable notification budget.

For **normal** custom permissions, Google already documents the cross-app exposure risk, but governance has not translated into enforcement. Build-time lint, store-review gates on **normal**-guarded sensitive exports, and a **signature** default for developer-defined permissions would reduce this surface without breaking legitimate interoperability. The fact that most recovered exploitation call sites originate from bundled third-party SDKs rather than first-party app code also suggests that remediation should not stop at app developers; it should extend to SDK vendors and to store policies that flag SDK-introduced **normal**-guarded exports during review. Questions that remain open, including how users interpret update-time notices at scale and when re-consent should be mandatory, require user studies and platform-policy experiments.

5.1 Limitations

The permission-group analysis is subject to one dataset-level qualification: Andro-Zoo aggregates APKs from multiple markets, and for 22.77% of the expanding apps the two consecutive versions we compared came from different markets. In those cases, an apparent expansion may reflect market-specific or regional variants rather than a user-observed in-place update; we therefore treat this subset explicitly as qualified and show separately that the effect persists in the Play-only and non-Play-only buckets. For the malware-association analysis, VirusTotal detection counts are a behavioral proxy rather than a ground-truth malware-family label, so our result is an association with aggregated VT flagging, not with any single malware family. For the custom-permission analysis, the 307-pair set is a static lower bound: reflection, dynamic loading, and native-code accesses are out of scope, and while manifest-level pair identification is obfuscation-stable, identifier renaming reduces requester-side call-site recovery. Exact field-level categorization is therefore available only for the 40 pairs whose exploitable-side `ContentProvider` exposes readable column-name strings, and our app-core versus library attribution is conservative because ProGuard repackaging can shift third-party code into the unclassified bucket. Finally, the 96-day Pixel 7 deployment of the transparency prototype is a feasibility study on a single device and install profile, not an ecosystem-scale estimate of notification burden.

6 Related Work

Usable Security and Consent Mechanisms. Early Android versions relied on install-time permission lists that users largely ignored. Felt et al. [28] showed that users rarely understood or acted on bulk permission requests, while Kelley et al. [35] found that even simplified permission displays failed to improve comprehension. These findings motivated Android’s transition to runtime dialogs in Android 6.0. However, subsequent work revealed persistent challenges: Felt et al. [29] demonstrated that users struggled to make informed decisions even with contextual prompts, and Wijesekera et al. [52] showed that user preferences for permission grants varied significantly based on context, with many grants later regretted. Research on permission decision-making has emphasized the gap between user mental models and actual system behavior. Almuhimedi et al. [4] found that users were unaware of how frequently apps accessed location data, leading to surprise when shown access logs. Wijesekera et al. [53] proposed dynamically adjusting permission grants based on user context, finding that static grants poorly matched user preferences. These studies frame permissions as *interfaces for decision-making* where incomplete cues and prompt fatigue shape behavior, providing context for our investigation of silent post-grant expansion.

Permission Evolution and Capability Drift. Longitudinal analyses have documented how Android’s permission architecture produces “capability drift,” gaps between what users believe they approved and what apps can subsequently do. Wei et al. [51] tracked permission evolution across Android versions, showing that apps accumulate permissions over time. Calciati et al. [26] demonstrated that

once any group member is granted, later additions are automatically approved, eroding users’ ability to track capability changes. Backes et al. [23] analyzed the Android framework’s permission enforcement, revealing inconsistencies between documented and actual behavior. Almomani and Al-Khayer [3] argued that grouping by functionality rather than risk leads users to conflate distinct harms, undermining risk-aware consent. Reardon et al. [48] documented how apps circumvent permission restrictions through side channels and covert data access, demonstrating that permission grants do not fully constrain app behavior. This literature establishes that Android’s design directly shapes user expectations and their ability to revise prior decisions, providing critical context for our investigation of silent intra-group expansion on Android 16.

Custom Permissions and Cross-App Access. Custom permissions introduce developer-defined access control that operates outside AOSP’s dangerous permission model. Tuncay et al. [50] showed that inconsistently defined or weakly protected custom permissions enable privilege-escalation attacks through confused deputy vulnerabilities. Li et al. [38] systematized fuzzing of permission enforcement paths, finding fragile logic around declaration, default protection levels, and runtime checks. Enck et al. [27] developed TaintDroid to track sensitive data flows across app boundaries, revealing how inter-app communication can leak private information. Gamba et al. [30, 31] demonstrated that `normal`-level custom permissions frequently guard exported components handling sensitive information, enabling cross-app data access without corresponding AOSP dangerous permissions. Their responsible disclosure to Google received acknowledgment but no remediation, with the platform citing openness concerns. These studies show that auto-granted custom permissions effectively become symbolic labels, disconnecting user expectations from actual access control outcomes. Our work extends this line by confirming these patterns persist in Android 16, dynamically validating cross-developer exploitation, and quantifying prevalence at ecosystem scale.

Wearable and IoT Companion-App Permission Studies. A related line of work studies permission-mediated exposure in wearable and IoT companion-app ecosystems. Mujahid et al. [42, 43] analyze permission mismatches across Wear apps, Tileria et al. [49] apply taint analysis to Wear OS companion apps, Yeke et al. [54] examine the Android/Wear OS runtime permission model, and Neupane et al. [44] extend this line to IoT mobile companion apps. The common focus in that literature is a same-developer companion ecosystem and information flow between phone and paired-device components. Our analysis instead targets cross-developer apps co-installed on a single Android device, where the relevant path is Android IPC via `ContentResolver.query`. Because the exploiting and exploitable apps never share an address space, there is no intra-process data flow to taint-track; the consent violation occurs at the IPC boundary the moment the unrelated app issues the query, so we verify reachability end-to-end via Probe Apps on Android 16 rather than inferring it from a companion-app flow graph. The threat model is therefore unauthorized cross-app reads enabled by within-

group auto-grant and `normal` custom permissions, rather than companion-app information flow across trusted endpoints.

7 Ethical Considerations and Responsible Disclosure

Our study identifies apps that expose sensitive data to unrelated developers through `normal`-level custom permissions; we withheld app identities and followed coordinated disclosure. Of the 307 cross-developer pairs, 102 exploitable apps are currently listed on Google Play and were reported by email to the Google Android Security Team; apps no longer on Play or only on third-party markets were reported by email through the same channel. We also emailed the developers of the exploitable apps whose providers returned concrete data-type indicators: 43 developers acknowledged receipt of our report, and 14 have since confirmed that a new build addresses the issue, with no provider now protected by a `normal` custom permission. At the time of writing, we have not received a substantive response from Google or a confirmed platform-level remediation. Dynamic testing ran on isolated researcher-owned devices with synthetic data; no sensitive user data was retained. The study involves no human subjects and no IRB review is applicable.

8 Conclusion

Android’s permission model is built on the premise that the user consents once and the platform enforces what was approved. Two long-documented mechanisms weaken that premise: permission groups silently broaden an app’s capabilities after the user has approved any one member of the group, and `normal`-level custom permissions grant any installed app access to a developer-defined permission with no user prompt and no surface in Settings to revoke it. Both mechanisms turn one-time consent into a standing authorisation that the user has no practical way to inspect, and on Android 16 they still expose contacts, messages, location, credentials, identity, and health records to unrelated developers, including on widely deployed and pre-installed software. This paper shows that returning informed consent to Android’s update path is possible from outside the operating system: a lightweight transparency layer built entirely on public Android APIs surfaces each silent expansion to the user at update time and lets them act on it.

Acknowledgments. We thank the anonymous reviewers for their comments. This work was supported in part by the National Science Foundation under grant CNS-2211576 and by a Google Android Security and Privacy Research (ASPIRE) Award.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

A Additional Permission-Group Expansion Flows

Table 4 reports the top permission-group expansion flows for the five groups not shown in the main-paper table.

Table 4: Top-6 permission-group expansion flows for the remaining five groups.

Group	Flow	Count	Group	Flow	Count
STORAGE	Write Ext. → Read Ext.	177,685	NEARBY DEVICES	BT Connect → BT Advert.	2,818
STORAGE	Write Ext. → Images	105,909	NEARBY DEVICES	BT Scan → BT Advert.	2,442
STORAGE	Read Ext. → Images	104,765	NEARBY DEVICES	BT Connect → BT Scan	1,933
STORAGE	Write Ext. → Video	66,054	NEARBY DEVICES	BT Scan → BT Connect	427
STORAGE	Read Ext. → Video	65,045	NEARBY DEVICES	BT Advert. → BT Connect	59
STORAGE	Read Ext. → Audio	45,117	NEARBY DEVICES	BT Advert. → BT Scan	48
LOCATION	Fine → Coarse	47,818	CALENDAR	Write → Read	839
LOCATION	Fine → Background	35,339	CALENDAR	Read → Write	262
LOCATION	Coarse → Background	33,451	SENSORS	Body → Body BG	49
LOCATION	Coarse → Fine	22,451	SENSORS	Body BG → Body	2
LOCATION	Background → Coarse	1,287			
LOCATION	Background → Fine	195			

B Per-Group Expansion Volume

Table 5 breaks down permission-group expansion volume by group across the 2,244,575 multi-version apps analysed in Section 3.2. Values are independent of the VT threshold; per-group malware concentration is discussed in the body.

Table 5: Permission-group expansion volume over 2,244,575 multi-version apps.

Permission Group	Expanding Apps	Expansions	% of Total
STORAGE	283,875	677,705	73.0
LOCATION	96,947	140,541	15.1
PHONE	43,682	58,976	6.4
CONTACTS	22,427	30,591	3.3
SMS	5,361	11,463	1.2
NEARBY_DEVICES	4,671	7,775	0.8
CALENDAR	1,087	1,101	0.1
CALL_LOG	234	249	0.0
SENSORS	51	51	0.0
Total	381,026	928,452	100.0

C Stratified Malware Association

Table 6 reports quartile-stratified odds ratios for the malware-expansion association when controlling for each app’s maximum declared permission count.

Table 6: Quartile-stratified malware odds ratios ($VT \geq 20$) for expansion status, controlling for total permission count.

Max Permission Quartile	Apps (n)	OR
Q1 (1–8)	682,493	0.93
Q2 (9–12)	451,029	0.78
Q3 (13–23)	576,347	0.70
Q4 (24+)	534,706	2.06
Pooled Mantel-Haenszel	2,244,575	1.05 [1.02, 1.08]

References

1. Akanji, O.A., Egele, M., Stringhini, G.: The cost of convenience: Identifying, analyzing, and mitigating predatory loan applications on Android. In: Proceedings of the ACM Asia Conference on Computer and Communications Security (2026)
2. Allix, K., Bissyandé, T.F., Klein, J., Le Traon, Y.: Androzoo: Collecting millions of android apps for the research community. In: Proceedings of the International Conference on Mining Software Repositories (2016)
3. Almomani, I.M., Khayer, A.A.: A comprehensive analysis of the android permissions system. IEEE Access (2020)
4. Almuhamdi, H., Schaub, F., Sadeh, N., Adjerid, I., Acquisti, A., Gluck, J., Cranor, L.F., Agarwal, Y.: Your Location has been Shared 5,398 Times!: A Field Study on Mobile App Privacy Nudging. In: Proceedings of the CHI Conference on Human Factors in Computing Systems (2015)
5. Androguard: Reverse engineering and pentesting for android applications (2014), <https://github.com/androguard/androguard>
6. Android Developer: Behavior changes: Apps targeting android 13 or higher (2024), <https://developer.android.com/training/data-storage/shared/media#storage-permission>
7. Android Developer: Define custom permissions (2024), <https://developer.android.com/guide/topics/permissions/defining>
8. Android Developers: Android 9 behavior changes: All apps - call_log permission group (2018), <https://developer.android.com/about/versions/pie/android-9.0-changes-all#restrict-non-sdk>
9. Android Developers: Access documents and other files from shared storage (2024), <https://developer.android.com/training/data-storage/shared/documents-files>
10. Android Developers: Access media files from shared storage (2024), <https://developer.android.com/about/versions/13/behavior-changes-13#granular-media-permissions>

11. Android developers: App permissions (2024), <https://developer.android.com/guide/topics/permissions/overview>
12. Android Developers: Behavior changes: Apps targeting android 13 or higher (2024), <https://developer.android.com/about/versions/13/behavior-changes-13>
13. Android Developers: Bluetooth permissions (2024), <https://developer.android.com/develop/connectivity/bluetooth/bt-permissions>
14. Android Developers: Grant partial access to photos and videos (2024), <https://developer.android.com/about/versions/14/changes/partial-photo-video-access>
15. Android Developers: Permissions overview: Permission groups (2024), <https://developer.android.com/guide/topics/manifest/permission-group-element.html>
16. Android Developers: Permissions updates in Android 11 (2024), <https://developer.android.com/about/versions/11/privacy/permissions>
17. Android Developers: Photo picker (2024), <https://developer.android.com/training/data-storage/shared/photopicker>
18. Android Developers: Retrieve a list of contacts (2024), <https://developer.android.com/training/contacts-provider/retrieve-names>
19. Android Developers Blog: Making permissions auto-reset available to billions more devices (2021), <https://android-developers.googleblog.com/2021/09/making-permissions-auto-reset-available.html>
20. Android Developers Blog: Choosing the right storage experience (2023), <https://android-developers.googleblog.com/2023/08/choosing-right-storage-experience.html>
21. Android Open Source Project: Runtime permissions (2024), https://source.android.com/docs/core/permissions/runtime_perms
22. Backes, M., Bugiel, S., Derr, E.: Reliable third-party library detection in Android and its security applications. In: Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (2016)
23. Backes, M., Bugiel, S., Derr, E., Hammer, C., McDaniel, P.: On demystifying the Android application framework: Re-visiting Android permission specification analysis. In: Proceedings of the USENIX Security Symposium (2016)
24. Blair, I.: Mobile app download statistics & usage statistics. BuildFire (2024), <https://buildfire.com/app-statistics/>
25. Book, T., Wallach, D.S.: A case of collusion: A study of the interface between ad libraries and their apps. In: Proceedings of the ACM Workshop on Security and Privacy in Smartphones and Mobile Devices (2013)
26. Calciati, P., Kuznetsov, K., Gorla, A., Zeller, A.: Automatically granted permissions in android apps: An empirical study on their prevalence and on the potential threats for privacy. In: Proceedings of the International Conference on Mining Software Repositories (2020)
27. Enck, W., Gilbert, P., Chun, B.G., Cox, L.P., Jung, J., McDaniel, P., Sheth, A.N.: TaintDroid: An information-flow tracking system for realtime privacy monitoring on smartphones. In: Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (2010)
28. Felt, A.P., Chin, E., Hanna, S., Song, D., Wagner, D.: Android permissions demystified. In: Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (2011)
29. Felt, A.P., Ha, E., Egelman, S., Haney, A., Chin, E., Wagner, D.: Android permissions: User attention, comprehension, and behavior. In: Proceedings of the Symposium on Usable Privacy and Security (2012)

30. Gamba, J., Álvaro Feal, Blázquez, E., Bandara, V., Razaghpanah, A., Tapiador, J.E., Vallina-Rodríguez, N.: Mules and permission laundering in android: Dissecting custom permissions in the wild. *IEEE Transactions on Dependable and Secure Computing* (2024)
31. Gamba, J., Rashed, M., Razaghpanah, A., Tapiador, J., Vallina-Rodríguez, N.: An Analysis of Pre-installed Android Software. In: *Proceedings of the IEEE Symposium on Security and Privacy* (2020)
32. Google Play Console Help: Use of SMS or Call Log permission groups (2024), <https://support.google.com/googleplay/android-developer/answer/9888170>
33. Google/Android: Android passes 3 billion monthly active devices (2025), <https://blog.google/products/android/the-android-show-io-2025/>
34. Grace, M.C., Zhou, W., Jiang, X., Sadeghi, A.R.: Unsafe exposure analysis of mobile in-app advertisements. In: *Proceedings of the ACM Conference on Security and Privacy in Wireless and Mobile Networks* (2012)
35. Kelley, P.G., Consolvo, S., Cranor, L.F., Jung, J., Sadeh, N., Wetherall, D.: A conundrum of permissions: Installing applications on an Android smartphone. In: *Proceedings of the International Conference on Financial Cryptography and Data Security* (2012)
36. Kollnig, K., Binns, R., Dewitte, P., Van Kleek, M., Wang, G., Omeiza, D., Webb, H., Shadbolt, N.: A Fait accompli? an empirical study into the absence of consent to third-party tracking in Android apps. In: *Proceedings of the Symposium on Usable Privacy and Security* (2021)
37. Li, M., Wang, W., Wang, P., Wang, S., Wu, D., Liu, J., Xue, R., Huo, W.: LibD: Scalable and precise third-party library detection in Android markets. In: *Proceedings of the International Conference on Software Engineering* (2017)
38. Li, R., Diao, W., Li, Z., Du, J., Guo, S.: Android Custom Permissions Demystified: From Privilege Escalation to Design Shortcomings. In: *Proceedings of the IEEE Symposium on Security and Privacy* (2021)
39. Liski, E.: An introduction to categorical data analysis, 2nd edition by alan agresti. *International Statistical Review* (2007)
40. Ma, Z., Wang, H., Guo, Y., Chen, X.: LibRadar: Fast and accurate detection of third-party libraries in Android apps. In: *Proceedings of the International Conference on Software Engineering Companion* (2016)
41. Mantel, N., Haenszel, W.M.: Statistical aspects of the analysis of data from retrospective studies of disease. *Journal of the National Cancer Institute* (1959)
42. Mujahid, S., Abdalkareem, R., Shihab, E.: Studying permission related issues in Android wearable apps. In: *Proceedings of the IEEE International Conference on Software Maintenance and Evolution* (2018)
43. Mujahid, S., Sierra, G., Abdalkareem, R., Shihab, E., Shang, W.: Detecting wearable app permission mismatches: A case study on Android wear. In: *Proceedings of the Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering* (2017)
44. Neupane, S., Tazi, F., Paudel, U., Baez, F.V., Adamjee, M., Carli, L.D., Das, S., Ray, I.: On the data privacy, security, and risk postures of iot mobile companion apps. In: *Database Security* (2022)
45. Nielsen, J.: Enhancing the explanatory power of usability heuristics. In: *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (1994)
46. Pearson, K.: On the criterion that a given system of deviations from the probable in the case of a correlated system of variables is such that it can be reasonably supposed to have arisen from random sampling (1992)

47. Play Console Help: Permissions and apis that access sensitive information (2024), <https://support.google.com/googleplay/android-developer/answer/16558241>
48. Reardon, J., Feal, Á., Wijesekera, P., On, A.E.B., Vallina-Rodriguez, N., Egelman, S.: 50 ways to leak your data: An exploration of apps' circumvention of the android permissions system. In: Proceedings of the USENIX Security Symposium (2019)
49. Tileria, M., Blasco, J., Suarez-Tangil, G.: WearFlow: Expanding information flow analysis to companion apps in wear OS. In: Proceedings of the Symposium on Research in Attacks, Intrusions and Defenses (2020)
50. Tuncay, G., Demetriou, S., Ganju, K., Gunter, C.: Resolving the predicament of android custom permissions. In: Proceedings of the Network and Distributed System Security Symposium (2018)
51. Wei, X., Gomez, L., Neamtiu, I., Faloutsos, M.: Permission evolution in the Android ecosystem. In: Proceedings of the Annual Computer Security Applications Conference (2012)
52. Wijesekera, P., Baokar, A., Hosseini, A., Egelman, S., Wagner, D., Beznosov, K.: Android permissions remystified: A field study on contextual integrity. In: Proceedings of the USENIX Security Symposium (2015)
53. Wijesekera, P., Baokar, A., Tsai, L., Reardon, J., Egelman, S., Wagner, D., Beznosov, K.: The feasibility of dynamically granted permissions: Aligning mobile privacy with user preferences. In: Proceedings of the IEEE Symposium on Security and Privacy (2017)
54. Yeke, D., Ibrahim, M., Tuncay, G.S., Farrukh, H., Imran, A., Bianchi, A., Celik, Z.B.: Wear's my data? Understanding the cross-device runtime permission model in wearables. In: Proceedings of the IEEE Symposium on Security and Privacy (2024)
55. Zhu, S., Shi, J., Yang, L., Qin, B., Zhang, Z., Song, L., Wang, G.: Measuring and modeling the label dynamics of online anti-malware engines. In: Proceedings of the USENIX Security Symposium (2020)